SharkHub Business Rules API

Postman Execution Routes Guide

Orden de creación, obtención de identificadores y validación por rol

Documento complementario para ejecutar la colección de Postman sin perder dependencias entre categorías.

Fuentes usadas: Swagger UI de la API, colección Postman, entorno Postman y archivo de elementos de validación. Los JWT se reemplazan por variables de entorno.

General Description

Este documento indica el orden correcto de ejecución en Postman para crear y conservar los elementos necesarios de SharkHub: owner, barbería, barbero, cliente, servicio, disponibilidad, cita, invitación y validaciones de dashboard. La utilidad principal es evitar confusiones cuando una ruta necesaria se encuentra en otra categoría de la colección, por ejemplo crear el barbero en Authentication & Sessions y luego asociarlo a la barbería desde Barbershops & Members.

La guía mantiene el mismo enfoque técnico de la documentación de endpoints, pero está organizada por flujo de trabajo. Cada paso indica la carpeta de Postman, el método, la ruta, el token requerido, el body y el identificador que se debe guardar para continuar.

1. Environment variables required in Postman

El entorno exportado incluye cuatro variables principales. Para este flujo se recomienda agregar variables adicionales de trabajo, porque varios endpoints requieren IDs creados previamente.

Variable	Type	Use
baseUrl	API host	URL base del backend de reglas de negocio. Ejemplo: http://<host>:3001
token	Bearer token	Token del owner. Usarlo como Authorization: Bearer {{token}}.
tokenBarber	Bearer token	Token del barbero. Usarlo en endpoints de barber dashboard o gestión autorizada.
tokenCustomer	Bearer token	Token del cliente. Usarlo para consultar servicios, horarios disponibles y agendar.

Recommended additional variables

Variable	Created / obtained from	Response field	Validation value used
owner_user_id	POST /auth/register or /auth/login	user.id	0a4dfa09-c93a-4c90-a4af-e775462367aa
shop_id	POST /barbershops	id	95214e64-98d5-4e39-82d4-487b8673ca90
owner_member_id	GET /barbershops/{shop_id}/members	member_id where role=owner	45594745-5dee-4e20-8e7d-8cc193c6ea07
barber_user_id	POST /auth/register	user.id	9d814758-fc3e-4da7-93c1-e6c5768c58c8
barber_member_id	POST /barbershops/{shop_id}/members	member_id	7929e68b-3b5d-438b-90ce-f52186f7df2b
customer_user_id	POST /auth/register or /auth/login	user.id	2e768092-e34e-47e9-9a1a-60bd4c207794
service_id	POST /barbershops/{shop_id}/services	service_id	5be23cc1-3fea-4f38-856c-d21d7bba607b
availability_id	POST /barbers/{barber_id}/availabilities	availability_id	c11da953-3d40-463e-95c0-f9abf379f053
appointment_id	POST /appointments or /api/customer/appointments	appointment_id	afd7e13f-829b-4c3c-b061-470a908eb3f9
invitation_id	POST /barbershops/{shop_id}/invitations	invitation_id	5684c9eb-b5c0-4f4f-9ce8-4fa60bfadbba

Important: the validation values are examples from the test run. In another database execution, Postman must save the new IDs returned by the API.

2. Dependency map

The main dependency chain is: owner account -> owner token -> barbershop -> owner membership -> barber account -> barber membership -> barber token -> customer account -> customer token -> service -> barber availability -> available times -> appointment -> dashboard validation.

Element	Postman folder	Route	Why it is needed
Owner user	Authentication & Sessions	POST /auth/register	Creates the account that later owns the barbershop.
Owner token	Authentication & Sessions	POST /auth/login	Required to create the barbershop and owner-only actions.
Barbershop	Barbershops & Members	POST /barbershops	Creates shop_id and automatically links the owner.
Barber user	Authentication & Sessions	POST /auth/register	Creates the user_id that will later become a barber member.
Barber member	Barbershops & Members	POST /barbershops/{shop_id}/members	Links barber_user_id to shop_id and produces member_id.
Customer user	Authentication & Sessions	POST /auth/register	Creates client_id / customer_user_id.
Service	Services	POST /barbershops/{shop_id}/services	Creates service_id used by appointments.
Availability	Barber Availabilities	POST /barbers/{barber_id}/availabilities	Creates schedule blocks for barber_user_id.
Appointment	Appointments or Customer Dashboard	POST /appointments or POST /api/customer/appointments	Creates appointment_id after validating service and availability.

ID rule summary

Field	Meaning	Recommended variable
barber_id	Use the barber user ID, not member_id.	{{barber_user_id}}
client_id	Use the customer user ID.	{{customer_user_id}}
member_id	Use only for member routes and owner status changes.	{{barber_member_id}}
shop_id	Use the id returned by POST /barbershops.	{{shop_id}}
service_id	Use the service_id returned by service creation/listing.	{{service_id}}
appointment_id	Use the appointment_id returned by appointment creation/listing.	{{appointment_id}}

3. Complete execution order

This section is the practical checklist for Postman. Execute the routes in the proposed order and save the response fields before moving to the next step.

#	Objective	Folder / method / route / auth	Save / result
0	Check API status	System / GET /health / No auth	Confirms backend availability.
1	Register owner	Authentication & Sessions / POST /auth/register / No auth	Save user.id as owner_user_id.
2	Login owner	Authentication & Sessions / POST /auth/login / No auth	Save token as token.
3	Create barbershop	Barbershops & Members / POST /barbershops / Bearer {{token}}	Save id as shop_id and invite_code if needed.
4	Confirm owner membership	Barbershops & Members / GET /barbershops/{shop_id}/members / Bearer {{token}}	Save owner member_id if needed.
5	Register barber user	Authentication & Sessions / POST /auth/register / No auth	Save user.id as barber_user_id.
6	Add barber to shop	Barbershops & Members / POST /barbershops/{shop_id}/members / Bearer {{token}}	Body requires user_id=barber_user_id. Save member_id.
7	Login barber	Authentication & Sessions / POST /auth/login / No auth	Save token as tokenBarber.
8	Register customer	Authentication & Sessions / POST /auth/register / No auth	Save user.id as customer_user_id.
9	Login customer	Authentication & Sessions / POST /auth/login / No auth	Save token as tokenCustomer.
10	Create service	Services / POST /barbershops/{shop_id}/services / Bearer {{token}} or {{tokenBarber}}	Save service_id.
11	Create availability	Barber Availabilities / POST /barbers/{barber_id}/availabilities / Bearer {{token}} or {{tokenBarber}}	Repeat for each day required. Save availability_id.
12	Search available times	Role Dashboards / GET /api/customer/available-times / Bearer {{tokenCustomer}}	Use query params: barber_id, service_id and date. Pick one returned slot.
13	Create appointment	Appointments or Customer Dashboard / POST /appointments or POST /api/customer/appointments / Bearer {{tokenCustomer}}	Save appointment_id.
14	Validate dashboards	Role Dashboards / /api/owner/*, /api/barber/*, /api/customer/* / correct role token	Use token according to owner, barber or customer.

4. Step-by-step route details

4.1 Owner account and token

POST `/auth/register`

Postman folder	Authorization	Purpose
Authentication & Sessions	No auth	Create owner user. Save response user.id as {{owner_user_id}}.

Request body

```
{
"name": "Owner SharkHub",
"email": "owner@example.com",
"password": "<password>"
}
```

POST `/auth/login`

Postman folder	Authorization	Purpose
Authentication & Sessions	No auth	Login owner. Save response token as {{token}}.

Request body

```
{
"email": "owner@example.com",
"password": "<password>"
}
```

GET `/sessions/current`

Postman folder	Authorization	Purpose
Authentication & Sessions	Bearer {{token}}	Optional check. Confirms that the owner session is active.

4.2 Barbershop and owner membership

POST `/barbershops`

Postman folder	Authorization	Purpose
Barbershops & Members	Bearer {{token}}	Create the barbershop. Save id as {{shop_id}}. The creator becomes owner automatically.

Request body

```
{
"name": "SharkHub Barbería",
"slug": "sharkhub-barberia",
"description": "Barbería principal de pruebas",
"logo_url": "logo.png",
"address": "Sangolquí",
"phone": "0987654321",
"email": "barberia@example.com"
}
```

GET `/barbershops/{shop_id}`

Postman folder	Authorization	Purpose
Barbershops & Members	No auth	Verify the barbershop data.

GET `/barbershops/{shop_id}/members`

Postman folder	Authorization	Purpose
Barbershops & Members	Bearer {{token}}	Verify the owner member and later the barber member.

4.3 Barber user and membership

POST `/auth/register`

Postman folder	Authorization	Purpose
Authentication & Sessions	No auth	Create the barber user. Save user.id as {{barber_user_id}}.

Request body

```
{
"name": "Carlos Barber",
"email": "barber1@example.com",
"password": "<password>"
}
```

POST `/barbershops/{shop_id}/members`

Postman folder	Authorization	Purpose
Barbershops & Members	Bearer {{token}}	Add the barber as an active member of the same barbershop. Save member_id as {{barber_member_id}}.

Request body

```
{
"user_id": "{{barber_user_id}}",
"role": "barber"
}
```

POST `/auth/login`

Postman folder	Authorization	Purpose
Authentication & Sessions	No auth	Login barber and save response token as {{tokenBarber}}.

Request body

```
{
"email": "barber1@example.com",
"password": "<password>"
}
```

4.4 Customer user and token

POST `/auth/register`

Postman folder	Authorization	Purpose
Authentication & Sessions	No auth	Create the customer user. Save user.id as {{customer_user_id}}.

Request body

```
{
"name": "Test Customer",
"email": "customer1@example.com",
"password": "<password>"
}
```

POST `/auth/login`

Postman folder	Authorization	Purpose
Authentication & Sessions	No auth	Login customer and save response token as {{tokenCustomer}}.

Request body

```
{
"email": "customer1@example.com",
"password": "<password>"
}
```

4.5 Service required for appointments

POST `/barbershops/{shop_id}/services`

Postman folder	Authorization	Purpose
Services	Bearer {{token}} or {{tokenBarber}}	Create service inside the same barbershop. Save service_id.

Request body

```
{
"barbershop_id": "{{shop_id}}",
"name": "Corte Normal",
"description": "Para cabellos normales",
"price": 5,
"duration_minutes": 10
}
```

GET `/barbershops/{shop_id}/services`

Postman folder	Authorization	Purpose
Services	No auth	List services of the shop and recover service_id if needed.

GET `/barbershops/{shop_id}/services/{service_id}`

Postman folder	Authorization	Purpose
Services	No auth	Verify the specific service.

4.6 Barber availability required before booking

POST `/barbers/{barber_id}/availabilities`

Postman folder	Authorization	Purpose
Barber Availabilities	Bearer {{token}} or {{tokenBarber}}	Create one availability block for the barber. Here barber_id must be {{barber_user_id}}.

Request body

```
{
"barbershop_id": "{{shop_id}}",
"day_of_week": 1,
"start_time": "07:00:00",
"end_time": "19:00:00",
"is_available": true
}
```

GET `/barbers/{barber_id}/availabilities`

Postman folder	Authorization	Purpose
Barber Availabilities	No auth	Verify created blocks. Use this route when appointment creation says the barber is not available.

4.7 Appointment creation

GET `/api/customer/available-times?barber_id={barber_id}&service_id={service_id}&date={date}`

Postman folder	Authorization	Purpose
Role Dashboards	Bearer {{tokenCustomer}}	Recommended check before booking. Select a slot returned by the API.

POST `/appointments`

Postman folder	Authorization	Purpose
Appointments	Bearer {{tokenCustomer}}	Direct business endpoint. Requires client_id in the request body.

Request body

```
{
"barber_id": "{{barber_user_id}}",
"client_id": "{{customer_user_id}}",
"barbershop_id": "{{shop_id}}",
"appointment_date": "2026-06-16",
"start_time": "12:06:00",
"end_time": "12:36:00",
"notes": "",
"service_id": "{{service_id}}"
}
```

POST `/api/customer/appointments`

Postman folder	Authorization	Purpose
Role Dashboards	Bearer {{tokenCustomer}}	Customer dashboard route. It uses the authenticated customer, so client_id is not sent.

Request body

```
{
"barbershop_id": "{{shop_id}}",
"barber_id": "{{barber_user_id}}",
"service_id": "{{service_id}}",
"appointment_date": "2026-06-18",
"start_time": "10:00:00",
"end_time": "10:30:00",
"notes": "Reserva de prueba"
}
```

GET `/appointments/{appointment_id}`

Postman folder	Authorization	Purpose
Appointments	Bearer {{tokenCustomer}} or {{token}}	Verify appointment data.

5. Availability and booking notes

For your validation set, the barber availability was created from 07:00:00 to 19:00:00 for day_of_week 1 to 5. When creating an appointment, the date must fall on a day that has an availability block and the time must be inside the block.

day_of_week	Meaning in the validation flow	Example block
1	First working day used in validation	07:00:00 - 19:00:00
2	Second working day used in validation	07:00:00 - 19:00:00
3	Third working day used in validation	07:00:00 - 19:00:00
4	Fourth working day used in validation	07:00:00 - 19:00:00
5	Fifth working day used in validation	07:00:00 - 19:00:00

Suggested Postman loop for availability

Repeat the same POST /barbers/{barber_id}/availabilities request changing only day_of_week from 1 to 5. After creating the blocks, execute GET /barbers/{barber_id}/availabilities and verify that the API returns the list of blocks.

```
[
{
"barbershop_id": "{{shop_id}}",
"day_of_week": 1,
"start_time": "07:00:00",
"end_time": "19:00:00",
"is_available": true
},
{
"barbershop_id": "{{shop_id}}",
"day_of_week": 2,
"start_time": "07:00:00",
"end_time": "19:00:00",
"is_available": true
},
{
"barbershop_id": "{{shop_id}}",
"day_of_week": 3,
"start_time": "07:00:00",
"end_time": "19:00:00",
"is_available": true
},
{
"barbershop_id": "{{shop_id}}",
"day_of_week": 4,
"start_time": "07:00:00",
"end_time": "19:00:00",
"is_available": true
},
{
"barbershop_id": "{{shop_id}}",
"day_of_week": 5,
"start_time": "07:00:00",
"end_time": "19:00:00",
"is_available": true
...
```

Common booking errors and where to validate

Message	Cause	Route/check
El barbero especificado no es un miembro activo de esta barbería.	The barber_user_id is not linked to shop_id, or the member is inactive.	GET /barbershops/{shop_id}/members and POST /barbershops/{shop_id}/members
El barbero no está disponible en este horario.	The appointment date/time does not match an availability block.	GET /barbers/{barber_id}/availabilities and GET /api/customer/available-times
Token de autorización faltante o malformado.	The request does not send Authorization: Bearer <token>.	Check the request Authorization tab and selected environment.
Solo el Propietario (Owner) de la barbería puede realizar esta acción.	The route requires the owner token, not barber or customer token.	Use {{token}} from owner login.

6. Lookup routes to recover each ID

Use this section when you already created data but need to recover IDs. These are the routes that help you find each element again without recreating it.

Needed ID	Route	Folder	How to identify it
owner_user_id	GET /users or GET /users/me	Users	Find the owner by email or use current profile after owner login.
shop_id	GET /barbershops or GET /barbershops/{shop_id}	Barbershops & Members	Find the barbershop by name, slug or email.
owner_member_id	GET /barbershops/{shop_id}/members	Barbershops & Members	Find row where role=owner and user_id=owner_user_id.
barber_user_id	GET /users or login barber	Users / Auth	Use user.id returned by barber registration/login.
barber_member_id	GET /barbershops/{shop_id}/members	Barbershops & Members	Find row where user_id=barber_user_id.
customer_user_id	GET /users or login customer	Users / Auth	Use user.id returned by customer registration/login.
service_id	GET /barbershops/{shop_id}/services	Services	Use service_id from the matching service.
availability_id	GET /barbers/{barber_id}/availabilities	Barber Availabilities	Use availability_id from the desired day_of_week.
appointment_id	GET /appointments?barbershop_id={shop_id}&barber_id={barber_id}&appointment_date={date}	Appointments	Use appointment_id from matching appointment.
invitation_id	GET /barbershops/{shop_id}/invitations	Invitations	Use invitation_id from the invitation list.

Useful route index

Category	Route	Use
Authentication	POST /auth/register	Create owner, barber or customer user.
Authentication	POST /auth/login	Generate token for owner, barber or customer.
Sessions	GET /sessions/current	Validate current token session.
Users	GET /users/me	Get profile of the authenticated user.
Barbershops	POST /barbershops	Create the shop and assign owner automatically.
Members	POST /barbershops/{shop_id}/members	Attach barber_user_id to a shop.
Services	POST /barbershops/{shop_id}/services	Create a service inside the shop.
Availability	POST /barbers/{barber_id}/availabilities	Create barber schedule blocks.
Appointments	POST /appointments	Create appointment through base appointment route.
Customer dashboard	POST /api/customer/appointments	Create appointment using customer token.
Owner dashboard	GET /api/owner/appointments	List appointments as owner.
Barber dashboard	GET /api/barber/appointments	List appointments as barber.

7. Dashboard validation routes

After the base data exists, validate the role-oriented endpoints with the correct token. The same physical data is reused, but the route changes depending on the dashboard.

7.1 Owner dashboard - use {{token}}

Method	Route	Purpose	Body / parameters
PUT	/api/owner/barbershop	Update own barbershop fields.	{"description":"...","phone":"..."}
POST	/api/owner/barbers	Assign a barber by email.	{"email":"barber1@example.com"}
PATCH	/api/owner/barbers/{member_id}/status	Activate/inactivate barber member.	{"status":"active"}
GET	/api/owner/appointments?date={date}&barber_id={barber_id}	List appointments filtered by date and barber.	No body
PATCH	/api/owner/appointments/{appointment_id}/status	Confirm or change appointment status.	{"status":"confirmed"}

7.2 Barber dashboard - use {{tokenBarber}}

Method	Route	Purpose	Body / parameters
GET	/api/barber/appointments	List appointments assigned to the authenticated barber.	No body
POST	/api/barber/services	Create service as barber in the shop.	{"barbershop_id": "{{shop_id}}", "name": "Corte Normal", "description": "Para cabellos normales", "price": 5, "duration_minutes": 10}
POST	/api/barber/products	Create product as barber in the shop.	{"barbershop_id": "{{shop_id}}", "name": "Crema de afeitar Taurus", "description": "Facilidad para afeitar", "price": 10, "stock": 10, "image_url": "image.png"}

7.3 Customer dashboard - use {{tokenCustomer}}

Method	Route	Purpose	Body / parameters
GET	/api/customer/services?barbershop_id={shop_id}	Search services available in the selected shop.	Uses shop_id.
GET	/api/customer/available-times?barber_id={barber_id}&service_id={service_id}&date={date}	List available time slots for the barber, service and date.	Uses barber_user_id and service_id.
POST	/api/customer/appointments	Book an appointment as the authenticated customer.	{"barbershop_id": "{{shop_id}}", "barber_id": "{{barber_user_id}}", "service_id": "{{service_id}}", "appointment_date": "2026-06-18", "start_time": "10:00:00", "end_time": "10:30:00", "notes": "Reserva de prueba"}

8. Minimal Postman variable update checklist

After each successful request, update the environment variables manually or through Postman tests. The following checklist shows the exact response field to copy.

Moment	Copy from response	Store as
After POST /auth/login as owner	Copy token	token
After POST /auth/register as owner	Copy user.id	owner_user_id
After POST /barbershops	Copy id	shop_id
After POST /auth/register as barber	Copy user.id	barber_user_id
After POST /barbershops/{shop_id}/members	Copy member_id	barber_member_id
After POST /auth/login as barber	Copy token	tokenBarber
After POST /auth/register as customer	Copy user.id	customer_user_id
After POST /auth/login as customer	Copy token	tokenCustomer
After POST /barbershops/{shop_id}/services	Copy service_id	service_id
After POST /barbers/{barber_id}/availabilities	Copy availability_id	availability_id
After POST /appointments or /api/customer/appointments	Copy appointment_id	appointment_id

Recommended Postman Tests snippets

These snippets are optional. They can be placed in the Tests tab of the corresponding request to avoid copying IDs manually.

Owner login - save token

```
const data = pm.response.json();
pm.environment.set('token', data.token);
pm.environment.set('owner_user_id', data.user.id);
```

Create barbershop - save shop_id

```
const data = pm.response.json();
pm.environment.set('shop_id', data.id);
```

Create service - save service_id

```
const data = pm.response.json();
pm.environment.set('service_id', data.service_id || data.id);
```

Create appointment - save appointment_id

```
const data = pm.response.json();
pm.environment.set('appointment_id', data.appointment_id || data.id);
```

Final recommendation

Keep the base data consistent: the service, availability, barber member and appointment must belong to the same shop_id. Most validation errors in this API happen when IDs from different barbershops are mixed.